

Product Requirements Document (PRD)

Project Name: TecTalk Roulette

****Platform Vision**:** An impromptu speaking and technical communication trainer designed specifically for software engineers.

****Author**:** Principal Technical Product Manager

1. Executive Summary & Problem Statement

1.1 Vision

To bridge the gap between technical competency (coding) and verbal articulation. Many software engineers, CSE graduates, and competitive programmers excel at technical problem-solving but struggle with verbalizing their thought processes, discussing system architectural trade-offs, and debug reasoning during live technical interviews. TecTalk is a gamified, impromptu practice platform that forces structured verbal pitches under realistic preparation and speaking constraints.

1.2 Target Persona

* **CSE Graduates / Entry-level Candidates**: Preparing for first-round technical screens and conceptual assessments.

* **Junior to Mid-Level Software Engineers**: Transitioning to roles that require active participation in system design reviews, production triage, and cross-team communication.

* **Competitive Programmers**: Excellent at algorithms but needing practice explaining complexity and mechanics.

2. Architecture & Hybrid Technical Design

2.1 System Architecture

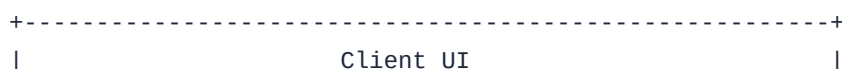
TecTalk employs a hybrid topology that balances low latency with real-time generative capabilities:

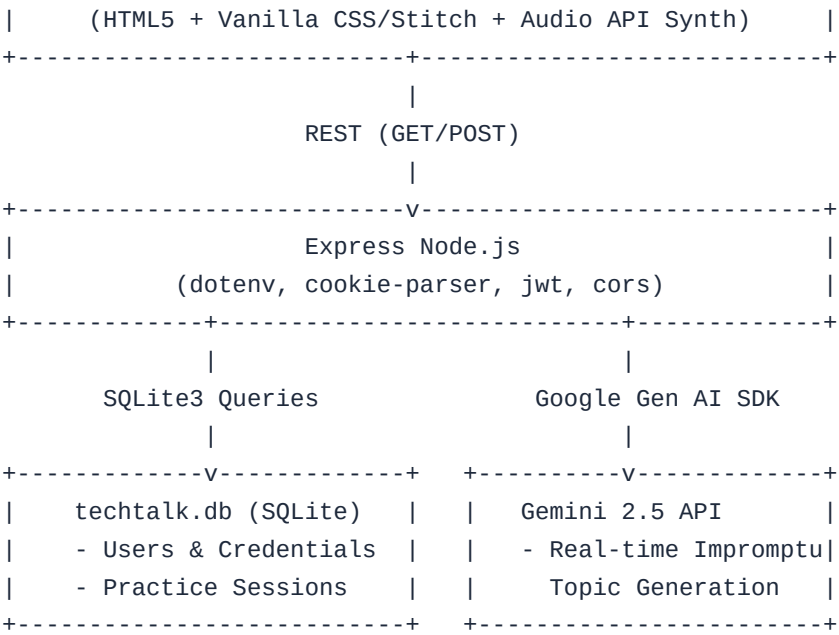
* **Dual Topic Engine**: Integrates a local database of 520 curated computer science questions with a dynamic Gemini API (gemini-2.5-flash) generation layer. The selection defaults to a 50/50 balance when an API key is configured.

* **Frontend**: Responsive vanilla HTML5 and modular JavaScript (`app.js`) designed around Google Stitch design tokens. Features glassmorphic panels, CSS Aurora glow effects, and Web Audio API synthesizers.

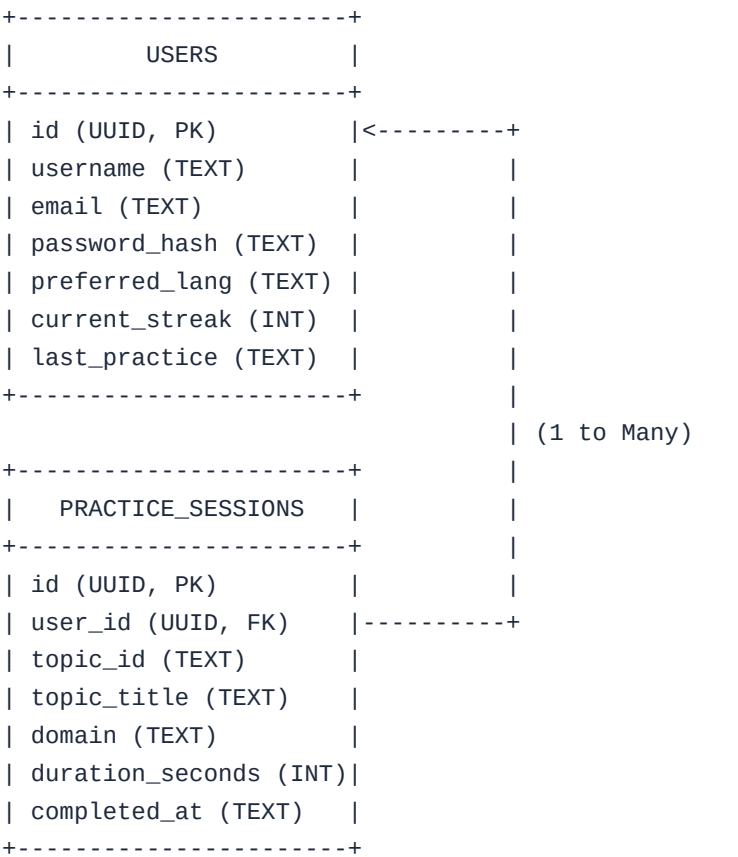
* **Backend**: Node.js Express server running ES Modules (ESM). Protects server credentials and communicates with Gemini using the `@google/genai` SDK.

* **Database**: SQLite3 database with tables for users, practice sessions, and tokens.





2.2 Entity Relationship Diagram (ERD)



3. Feature Specifications & Functional Requirements

3.1 Topic Generation & Slot-Machine Wheel Shuffle

* **Requirements**:

- The UI must render a slot-machine-style shuffle animation cycling through temporary topics when "SPIN TOPIC" is clicked.
- While spinning, the client queries the `/api/practice/spin`` GET endpoint to pre-fetch the final topic.
- If Gemini API is configured, the server uses a 50% probability to generate a new structured topic via ``gemini-2.5-flash`` with a JSON schema constraint (``id``, ``title``, ``domain``, ``prompt_guide``, ``key_research_points``).
- If API is missing or fails, it falls back to the 520 curated topics from ``cs_interview_topics_500.json``.
- The animation decelerates and lands precisely on the generated or retrieved topic.

3.2 10-Minute Preparation & Speaking Pitch Timers

* **Requirements**:

- Landing on a topic automatically switches the view to the Prep View and starts a 10:00 prep countdown.
- Buttons: "Spin Again" (cancels current timer and re-spins), "Finish Prep & Start Pitch" (forfeits prep and starts pitch).
- Transition: Reaching ``00:00`` or clicking "Finish Prep & Start Pitch" shifts to Speaking View.
- Speaking View hosts a real-time incrementing stopwatch measuring the length of the pitch, accompanied by a visual scratchpad editor for notes.

3.3 Calendar-Day Streak Engine

* **Requirements**:

- Evaluation: Streak validity is evaluated in real-time when the app loads.
- Reset: If ``last_practice_date`` is older than yesterday, ``current_streak`` is immediately reset to ``0`` and persisted.
- Increment: The streak increments by ``1`` only upon completing a speech session. Subsequent completions on the same calendar day do not increment it.
- Display: Bumps the streak badge value with a scale pop animation (``.streak-bounce``).

3.4 Web Audio API Audio Engine

* **Requirements**:

- Ticks: Real-time slot wheel tick sounds synthesized using a short oscillator tone.
- Landing: A warm chime synthesized upon topic landing (high frequency decay).
- Completion: Synthesized win chime on session completion.

3.5 Navigation Interceptor & Session Guard

* **Requirements**:

- Intercept tab triggers (``mastery``, ``history``, ``analytics``, ``settings``).
- If a practice session is active, block routing and prompt with ``#session-confirm-modal``.
- Stay: Keeps timer running. Leave: Resets state, clears timer, and switches tabs.
- Register ``beforeunload`` to block browser refresh during active sessions.

4. Non-Functional Requirements

4.1 Latency

- * The local spin draws and fallbacks must return within 100ms.

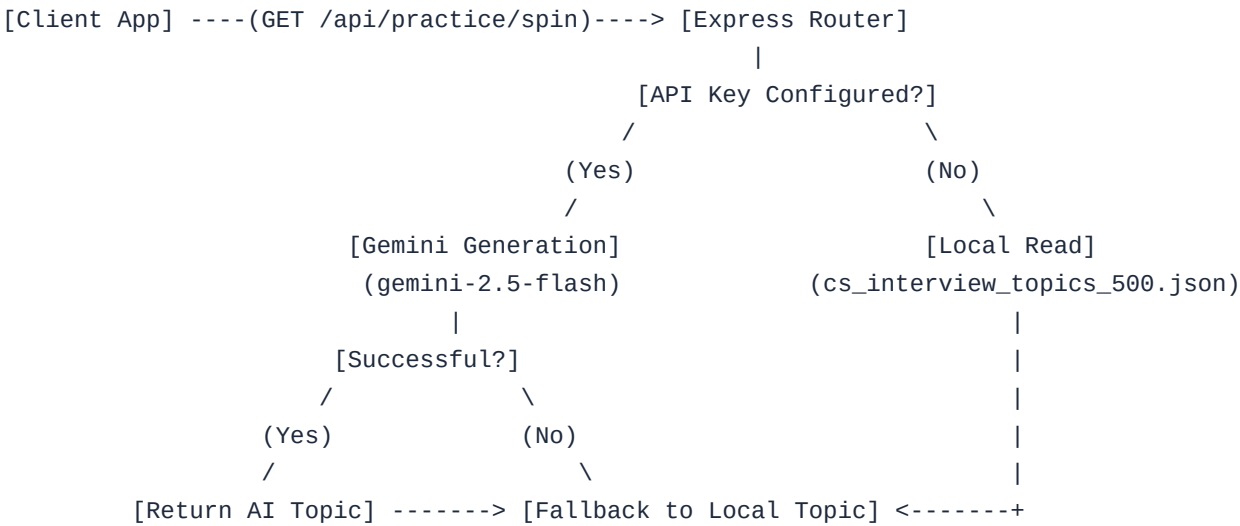
* API calls to Gemini must have a timeout mechanism (max 4.5 seconds) to trigger fallback to the local JSON dataset seamlessly.

4.2 Security

- * The Gemini API Key must be kept on the backend inside `.env`` and never exposed to the client.
- * Standard JWT token cookies with ``httpOnly`` and ``sameSite`` policies protect user sessions.
- * All input values on signup and profile editing are sanitized to prevent XSS.

5. Visual State Machine & Architecture Flow

System Architecture Flow



UI Workflow State Machine

